

PROYECTO FITNESS GYM

Grado Superior DAM



Módulo:Entornos de Desarrollo

Documento: Control de Calidad del Software con SonarQube

Alumno: Miguel Fandos Cuadra

Centro: CPIFP Bajo Aragón

Curso: 2025-2026

Fecha: 26/04/2026

Índice

1. Introducción	3
2. Configuración del entorno	3
3. Escaneos realizados por sprint	4
Sprint 1	4
Sprint 2	4
Sprint 3	4
Sprint Final	4
4. Problemas detectados	5
5. Correcciones planteadas	5
6. Resultado Final	6
7. Conclusión	6

1. Introducción

En este documento se recoge el uso de la herramienta SonarQube para realizar el análisis de calidad del software desarrollado en el proyecto Fitness Gym.

Se han realizado distintos análisis relacionados con los sprints del proyecto, con el objetivo de detectar incidencias, mejorar la calidad del código y verificar el cumplimiento de los criterios de calidad exigidos.

2. Configuración del entorno

Para la realización del análisis se ha configurado el siguiente entorno:

- Java 21 (Temurin)
- SonarQube Community
- SonarScanner CLI

Se realizó la creación del proyecto, generación del token de análisis y ejecución del escaneo desde la raíz del proyecto Java.

▲ Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine.

 **SonarQube**
community

Projects Issues Rules Quality profiles Quality gates Administration More ▾

1 of 2

Create a local project

Project display name * ⓘ

Project key * ⓘ

Main branch name *

The name of your project's default branch [Learn More](#) ⓘ

Cancel Next

3. Escaneos realizados por sprint

Sprint 1

Configuración inicial de SonarQube y SonarScanner.
Primer análisis del código para detección inicial de incidencias.

Sprint 2

Nuevo análisis tras ampliación del desarrollo del proyecto y revisión de problemas detectados.

Sprint 3

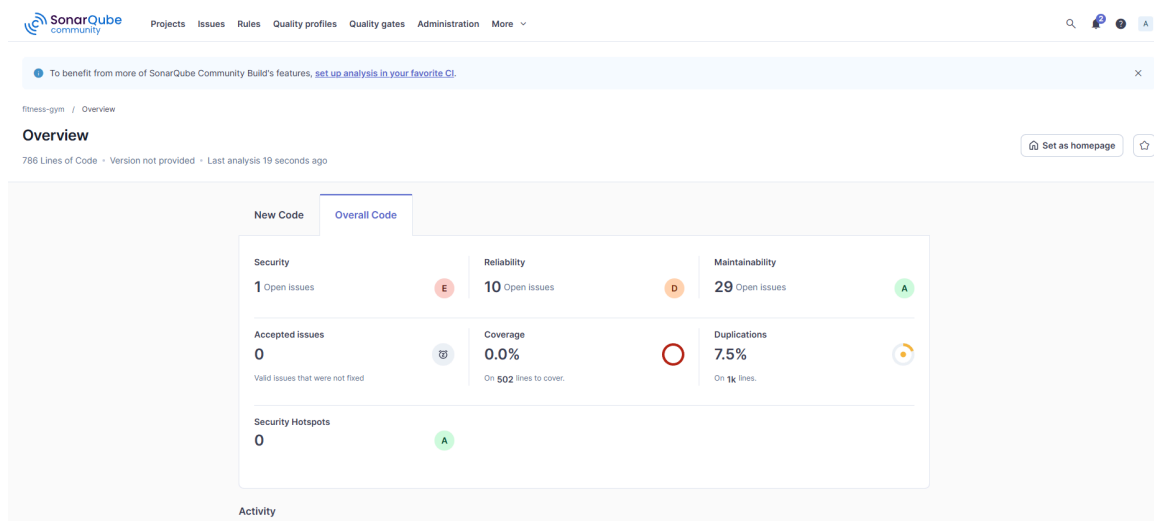
Análisis del código con mayor volumen de implementación y detección de incidencias de mantenibilidad y fiabilidad.

Sprint Final

Escaneo completo del proyecto.

Resultados obtenidos:

- 786 líneas de código analizadas
- 1 incidencia de seguridad
- 10 incidencias de fiabilidad
- 29 incidencias de mantenibilidad
- Duplicación detectada: 7.5%



4. Problemas detectados

Durante el análisis SonarQube detectó incidencias relacionadas con:

- Seguridad (protección de base de datos).
- Fiabilidad del código.
- Problemas de mantenibilidad.
- Uso de prácticas mejorables, como el empleo de `System.out` en lugar de un logger.

The screenshot displays the SonarQube web interface. At the top, a yellow banner contains a warning: "Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. [Learn more](#)". Below this, the navigation bar includes "SonarQube community", "Projects", "Issues", "Rules", "Quality profiles", "Quality gates", "Administration", and "More". The main content area is titled "My Issues" and "All". On the left, a "Filters" sidebar shows "Software Quality" with counts for Security (1), Reliability (10), and Maintainability (29). Under "Severity", it lists Blocker (1), High (15), Medium (18), Low (6), and Info (0). The main panel shows a list of issues for the project "fitness-gym / src/conexion/ConexionBD.java". The first issue is "Add password protection to this database." with a severity of "Security" and a "Blocker" status. The second and third issues are "Replace this use of System.out by a logger." with a severity of "Maintainability" and a "Medium" status. Each issue entry includes a checkbox, a description, a severity level, a status (e.g., "Not assigned"), and a link to "Open".

5. Correcciones planteadas

A partir del análisis realizado con SonarQube se identifican como acciones de mejora las siguientes:

- Incorporar protección o configuración segura en la conexión con la base de datos.
- Sustituir usos de `System.out` por un sistema de logging adecuado.
- Reducir duplicación de código detectada en el análisis.
- Revisar incidencias clasificadas como fiabilidad y mantenibilidad.
- Refactorizar partes del código para mejorar calidad y reducir incidencias detectadas.

6. Resultado Final

Se ha realizado correctamente el escaneo del proyecto mediante SonarQube, obteniendo métricas de calidad del código y detección de incidencias que permiten evaluar el estado del software desarrollado.

El análisis realizado ha permitido identificar incidencias relacionadas con seguridad, fiabilidad, mantenibilidad y duplicación de código, proporcionando información relevante para plantear mejoras y correcciones sobre el proyecto.

El uso de SonarQube ha permitido aplicar control de calidad al proyecto y verificar el estado del código dentro del proceso de desarrollo del proyecto.

7. Conclusión

La utilización de SonarQube ha permitido incorporar técnicas de análisis estático de código y control de calidad dentro del desarrollo del proyecto, detectando incidencias y proporcionando información para la mejora del software.